

# CMPM125 - Assignment 4 Report

## Team Members:

### Beckham Reyes:

Email: beareyes@ucsc.edu

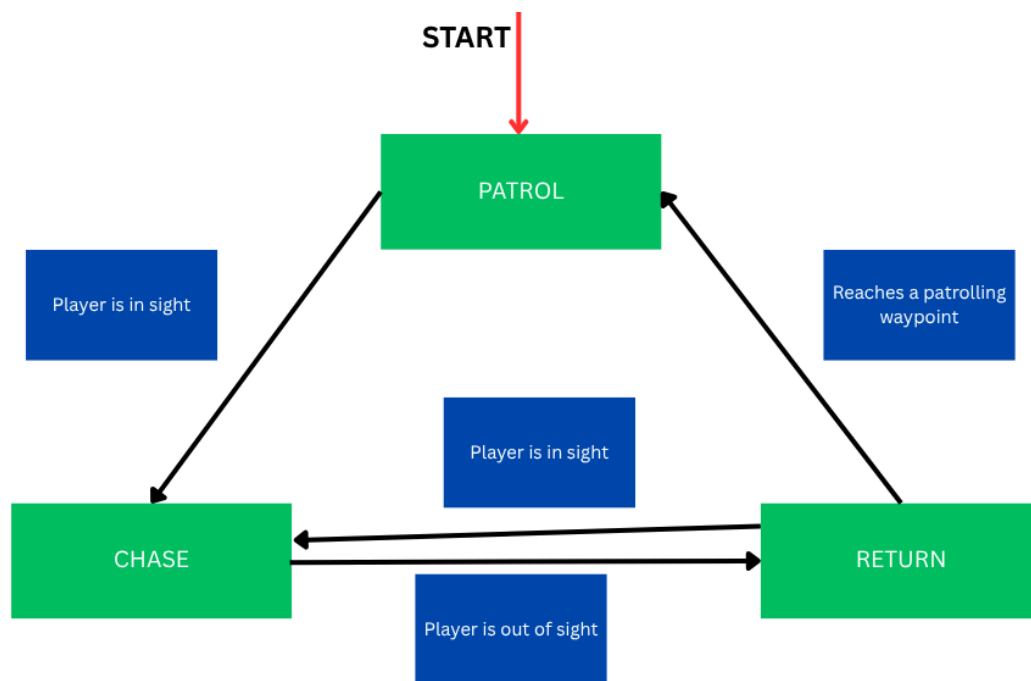
GitHub: BeReyes1

### Anish Bansal

Email: abansal6@ucsc.edu

GitHub: anish888-coder

## State Machine Diagram:



## AI Implementation:

The AI was split into four ideas being the Enemy, IState scripts, StateHandler, and StateMachine. StateMachine is a C# class that sets the state and handles the transitions via a dictionary. It allows types of state transitions which are from one state to another or any state to a specific state. The StateHandler defines the transitions. The two types of state transitions are seen here with At and Any and are triggered by a boolean condition:

```
//Transitions defined
Any(chase, () => enemy.PlayerInSight);
At(chase, returnState, () => !enemy.PlayerInSight);
At(returnState, patrol, () => returnState.ReachedWaypoint);
```

```
void At(IState from, IState to, Func<bool> condition) => stateMachine.AddTransition(from, to, condition);
void Any(IState to, Func<bool> condition) => stateMachine.AddAnyTransition(to, condition);
```

This is more versatile as we don't have to be redundant and define Patrol->Chase and Return->Chase when we can just do Any.

The StateHandler is a MonoBehaviour which allows us to initialize the Enemy it is attached to and put a gizmos which is good for debugging.

IState is an interface that has an enter, exit, update, and color functions. Exit ended up being unused but could be used in the future if the AI was expanded upon. Enter sets the color of the Enemy and sets the destination for the patrol and return states. Patrol and chase use Tick, the update function, because their destinations are changing unlike the Return state.

The Enemy state defines the waypoints and color of the enemy. It also tracks whether it sees the enemy and when true the chase state will be fired as defined via our StateHandler.

## Reflection:

### Beckham Reyes:

I created the enemy logic. This includes the state machine structure and logic for each state. I created the win logic and a restartable game.

I have a familiarity with state machines as I have used them for various AI in games from a shooter to a boxing game. I feel this is the best state machine I've created because I kept the logic mostly contained in each individual state.

### Anish Bansal:

I designed the level design. This involves the Navmeshing, the placements/sizing/ structuring,

assigning waypoints and the movable Navmeshobject. This is the first time ever working with vision cone detection and state changes.